

《嵌入式系统》第三次作业

基于 Qt (Quick time) 两次实验项目, 参考 Qt 指导手册, 完成下述任务:

1. 针对第一次 Qt 实验中的下述工程:

a) 自定义控件

i. 各文件关系

该工程文件及各自职责如下:

Circularprogressbar.pro: 构建配置, 列出源码与模块依赖 (Qt Widgets 等), 驱动 qmake 生成 Makefile。

circularprogressbar.h: 自定义控件类声明, 继承 QWidget, 声明计时器、绘制与事件处理接口。

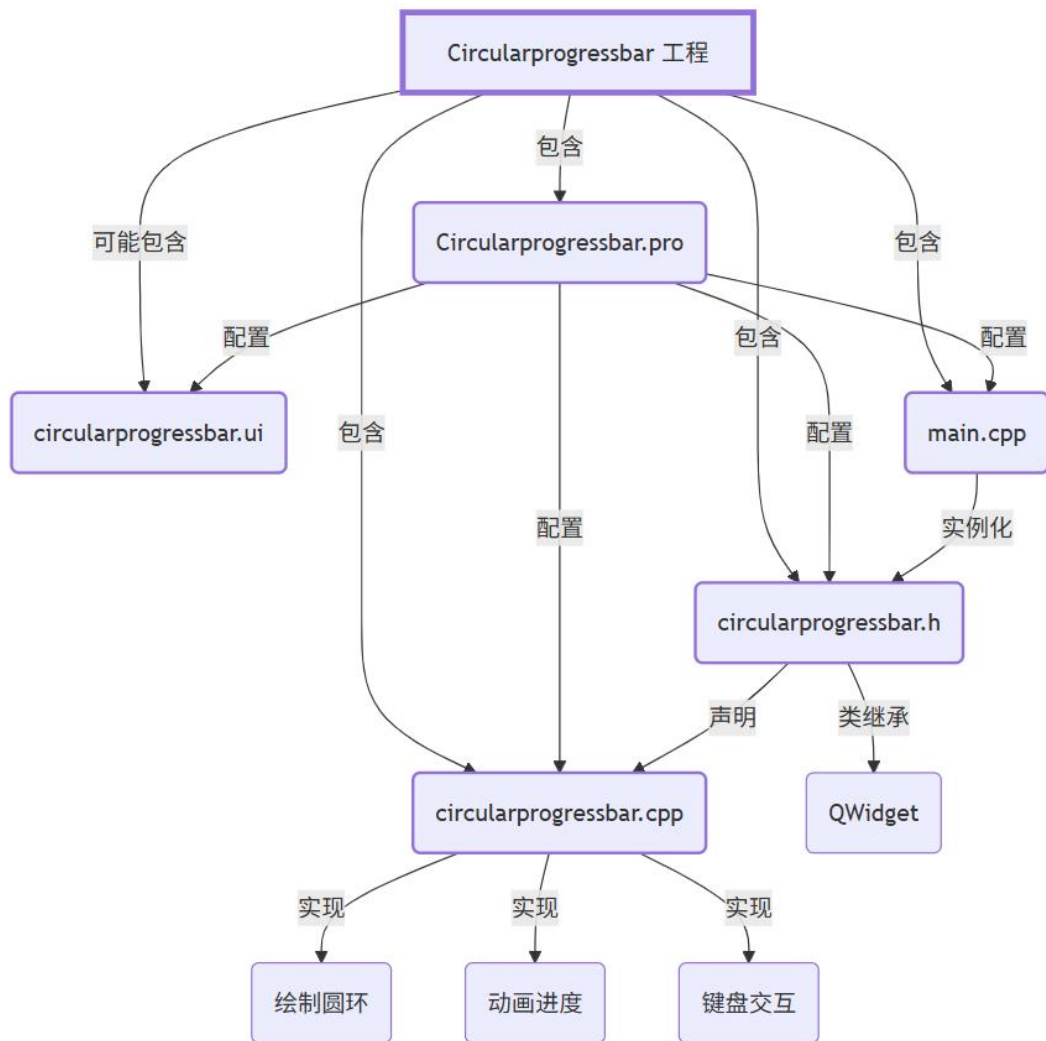
circularprogressbar.cpp: 实现类逻辑, 负责绘制圆环、动画进度、键盘交互。

circularprogressbar.ui: 若被使用则用于 Qt Designer 设计界面, 在本实验没有用它。

main.cpp: 程序入口, 创建 QApplication, 实例化并显示 Circularprogressbar。

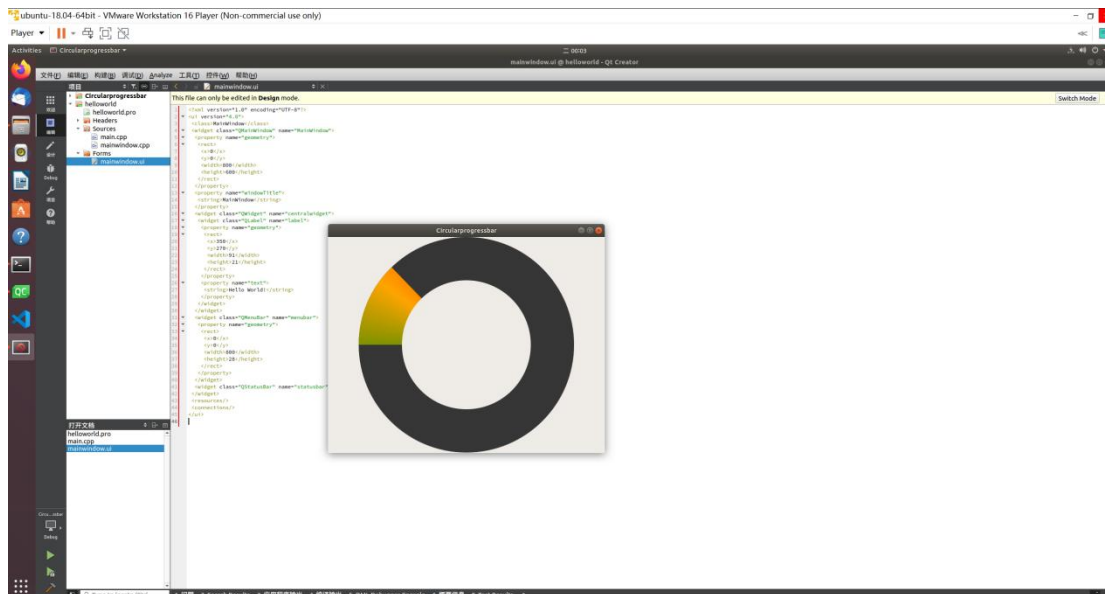
底下还生成了 pro.user 文件, 是 Qt Creator IDE 为当前用户该工程生成的个性化设置文件, 在剖析时就不纳入考虑了。

它们的关系图示如下, 使用 mermaid 绘制:



ii. 实验现象和原理

运行文件后弹出窗口，会有一个圆环进度条在窗口中心，按下空格增加进度、释放空格减少进度，如下图：



首先，进度条的“圆形”和“条状”是怎么实现的，通过代码我们可以看到，其分别绘制大圆、小圆，大圆从左边开始绘制彩色扇形，用小圆的颜色（跟背景一致）覆盖大圆，就形成了一个彩色的圆环。此逻辑在重绘事件函数 `paintEvent` 中体现：

```
58 // 重绘事件：分层绘制背景圆 -> 彩色进度 -> 内圆
59 void Circularprogressbar::paintEvent(QPaintEvent *event) {
60     Q_UNUSED(event);
61     QPainter painter(this);
62     int width = this->width();
63     int height = this->height();
64     painter.translate(width / 2, height / 2); // 坐标系移到中心
65     painter.setRenderHint(QPainter::Antialiasing, true); // 抗锯齿
66     painter.setPen(Qt::NoPen);
67
68     int outerRadius = std::min(width, height) / 2; // 外圆半径
69     int innerRadius = outerRadius - 50; // 内圆参考半径（用于计算缩小后内圆
70
71     drawBiggestCircle(painter, outerRadius); // 背景层
72     drawColor(painter, outerRadius); // 当前进度弧
73     drawLittleCircle(painter, innerRadius); // 内圆覆盖 -> 形成环
74 }
```

可以看到重绘函数调了 `drawBiggestCircle`、`drawColor`、`drawLittleCircle` 这三个函数。外、内圆的具体绘制：

```

12 // 绘制外圈底层大圆（背景暗灰）
13 void CircularProgressbar::drawBiggestCircle(QPainter &painter, int radius) {
14     painter.save();
15     QPainterPath path;
16     path.addEllipse(-radius, -radius, 2 * radius, 2 * radius); // 以当前中心为原点绘制
17     painter.setBrush(QColor(54, 54, 54)); // 暗灰填充作为底色
18     painter.drawPath(path);
19     painter.restore();
20 }
21
22 // 绘制内圈圆：通过缩小半径形成环结构
23 void CircularProgressbar::drawLittleCircle(QPainter &painter, int radius) {
24     painter.save();
25     QPainterPath path;
26     int reducedRadius = radius - 50; // 控制环厚度：外半径与此减小值差为厚度
27     path.addEllipse(-reducedRadius, -reducedRadius,
28                     2 * reducedRadius, 2 * reducedRadius);
29     QColor ringColor = palette().color(QPalette::Window); // 使用当前窗口背景色填充内圆
30     painter.setBrush(ringColor);
31     painter.drawPath(path);
32     painter.restore();
33 }

```

绘制颜色的实现：

```

35 // 绘制彩色进度弧：使用圆锥渐变 + drawPie 截取进度扇形
36 void CircularProgressbar::drawColor(QPainter &painter, int radius)
37 {
38     QRect rect(-radius, -radius, 2 * radius, 2 * radius); // 进度扇形的包围矩形
39     QConicalGradient Conical(0, 0, 0); // 圆心(0,0)，起始角度 0°
40
41     // 设置渐变色段：0 与 1 位置颜色相同以保证闭合连续
42     Conical.setColorAt(0.0, QColor(128, 0, 255)); // 深紫
43     Conical.setColorAt(0.05, QColor(128, 0, 255));
44     Conical.setColorAt(0.2, QColor(255, 0, 0)); // 红
45     Conical.setColorAt(0.4, QColor(255, 165, 0)); // 橙
46     Conical.setColorAt(0.6, QColor(0, 128, 0)); // 绿
47     Conical.setColorAt(0.8, QColor(0, 255, 255)); // 青
48     Conical.setColorAt(0.95, QColor(0, 0, 255)); // 蓝
49     Conical.setColorAt(1.0, QColor(128, 0, 255)); // 回到紫色
50
51     painter.setBrush(Conical);
52     // drawPie 角度单位为 1/16 度；起始角 -180*16 将起点放在圆左侧
53     // currentColorProgress 表示当前填充的角度（度），转换为 1/16 度需乘 16
54     painter.drawPie(rect, -180 * 16, -(currentColorProgress * 16));
55     // 使用负 spanAngle 是为了按顺时针方向（Qt 使用逆时针为正）
56 }

```

动画的驱动，则是通过每次定时器触发，根据 direction 布尔值决定增加或减少 currentColorProgress。每次增长步长: 3.6，回退步长: 1，这也是为什么进度条的回退比增长看起来比较慢。

到达边界 0 或 360 时钳制，并在 0 处停止计时器。

```

97 // 定时器槽：根据方向增减当前进度角度值并触发刷新
98 void Circularprogressbar::decreaseColorProgress()
99 {
100     if(direction){
101         currentColorProgress += 3.6; // 增长步长：一圈 360 / 3.6 = 100 次
102         if(currentColorProgress > 360){
103             currentColorProgress = 360; // 上限钳制
104         }
105     }else{
106         currentColorProgress -= 1; // 回退更慢：需 360 次清空
107         if(currentColorProgress < 0){
108             currentColorProgress = 0;
109             myTimer->stop(); // 清空后停止计时器节省资源
110         }
111     }
112     update(); // 触发 repaint -> paintEvent
113 }
114

```

其中定时器 myTimer 的定义在构造函数处初始化

```

3 // 构造函数：初始化计时器并建立信号槽连接
4 Circularprogressbar::Circularprogressbar(QWidget *parent)
5     : QWidget(parent)
6 {
7     myTimer = new QTimer(this); // 创建定时器，用于驱动颜色进度动画
8     connect(myTimer, &QTimer::timeout, // 每当定时器超时触发
9             this, &Circularprogressbar::decreaseColorProgress);
10 }
11

```

定时器的触发则是通过监视键盘按下和释放的事件，其具体函数如下：

```

76 // 键盘按下事件：空格开始增长方向并启动定时器
77 void Circularprogressbar::keyPressEvent(QKeyEvent *event)
78 {
79     if(event->key() == Qt::Key_Space)
80     {
81         myTimer->start();
82         direction = true; // 标记为增长模式
83         update(); // 立即请求重绘
84     }
85 }
86
87 // 键盘释放事件：切换为回退方向，继续由计时器驱动减色
88 void Circularprogressbar::keyReleaseEvent(QKeyEvent *event)
89 {
90     if(event->key() == Qt::Key_Space)
91     {
92         direction = false; // 切换为减少模式
93         update();
94     }
95 }
96

```

里面通过一个 bool 变量 direction 来标记进度条是增长还是减少的状态。

还能注意到，定时器槽、键盘事件函数的最后都调用了 update（）函数用于异步刷新，这个是继承自 QWidget 的函数，可以合并多次调用，避免频繁绘制。

iii. 核心代码文件注解

```
#include "circularprogressbar.h"

// 构造函数：初始化计时器并建立信号槽连接
Circularprogressbar::Circularprogressbar(QWidget *parent)
    : QWidget(parent)
{
    myTimer = new QTimer(this); // 创建定时器，用于驱动颜色进度动画
    connect(myTimer, &QTimer::timeout, // 每当定时器超时触发
            this, &Circularprogressbar::decreaseColorProgress);
}

// 绘制外圈底层大圆（背景暗灰）
void Circularprogressbar::drawBiggestCircle(QPainter &painter, int radius) {
    painter.save();
    QPainterPath path;
    path.addEllipse(-radius, -radius, 2 * radius, 2 * radius); // 以当前中心为原点绘制
    painter.setBrush(QColor(54, 54, 54)); // 暗灰填充作为底色
    painter.drawPath(path);
    painter.restore();
}

// 绘制内圈圆：通过缩小半径形成环结构
void Circularprogressbar::drawLittleCircle(QPainter &painter, int radius) {
    painter.save();
    QPainterPath path;
    int reducedRadius = radius - 50; // 控制环厚度：外半径与此减小值差为厚度
    path.addEllipse(-reducedRadius, -reducedRadius,
                    2 * reducedRadius, 2 * reducedRadius);
    QColor ringColor = palette().color(QPalette::Window); // 使用当前窗口背景色填充内圆
    painter.setBrush(ringColor);
    painter.drawPath(path);
    painter.restore();
}

// 绘制彩色进度弧：使用圆锥渐变 + drawPie 截取进度扇形
void Circularprogressbar::drawColor(QPainter &painter, int radius)
{
    QRect rect(-radius, -radius, 2 * radius, 2 * radius); // 进度扇形的包围矩形
    QConicalGradient Conical(0, 0, 0); // 圆心(0,0)，起始角度 0°
    // 设置渐变色段：0 与 1 位置颜色相同以保证闭合连续
    Conical.setColorAt(0.0, QColor(128, 0, 255)); // 深紫
    Conical.setColorAt(0.05, QColor(128, 0, 255));
    Conical.setColorAt(0.2, QColor(255, 0, 0)); // 红
```

```

        Conical.setColorAt(0.4, QColor(255, 165, 0)); // 橙
        Conical.setColorAt(0.6, QColor(0, 128, 0)); // 绿
        Conical.setColorAt(0.8, QColor(0, 255, 255)); // 青
        Conical.setColorAt(0.95, QColor(0, 0, 255)); // 蓝
        Conical.setColorAt(1.0, QColor(128, 0, 255)); // 回到紫色
        painter.setBrush(Conical);

        // drawPie 角度单位为 1/16 度; 起始角 -180*16 将起点放在圆左侧
        // currentColorProgress 表示当前填充的角度(度), 转换为 1/16 度需乘 16
        painter.drawPie(rect, -180 * 16, -(currentColorProgress * 16));

        // 使用负 spanAngle 是按顺时针方向
    }

// 重绘事件: 分层绘制背景圆 -> 彩色进度 -> 内圆
void Circularprogressbar::paintEvent(QPaintEvent *event) {
    Q_UNUSED(event);

    QPainter painter(this);

    int width = this->width();
    int height = this->height();

    painter.translate(width / 2, height / 2); // 坐标系移到中心
    painter.setRenderHint(QPainter::Antialiasing, true); // 抗锯齿
    painter.setPen(Qt::NoPen);

    int outerRadius = std::min(width, height) / 2; // 外圈半径
    int innerRadius = outerRadius - 50; // 内圈参考半径(用于计算缩小后内圆)
    drawBiggestCircle(painter, outerRadius); // 背景层
    drawColor(painter, outerRadius); // 当前进度弧
    drawLittleCircle(painter, innerRadius); // 内圆覆盖 -> 形成环
}

// 键盘按下事件: 空格开始增长方向并启动定时器
void Circularprogressbar::keyPressEvent(QKeyEvent *event)
{
    if(event->key() == Qt::Key_Space)
    {
        myTimer->start();

        direction = true; // 标记为增长模式
        update(); // 立即请求重绘
    }
}

// 键盘释放事件: 切换为回退方向, 继续由计时器驱动减色
void Circularprogressbar::keyReleaseEvent(QKeyEvent *event)
{
    if(event->key() == Qt::Key_Space)
    {
        direction = false; // 切换为减少模式
        update();
    }
}

```

```

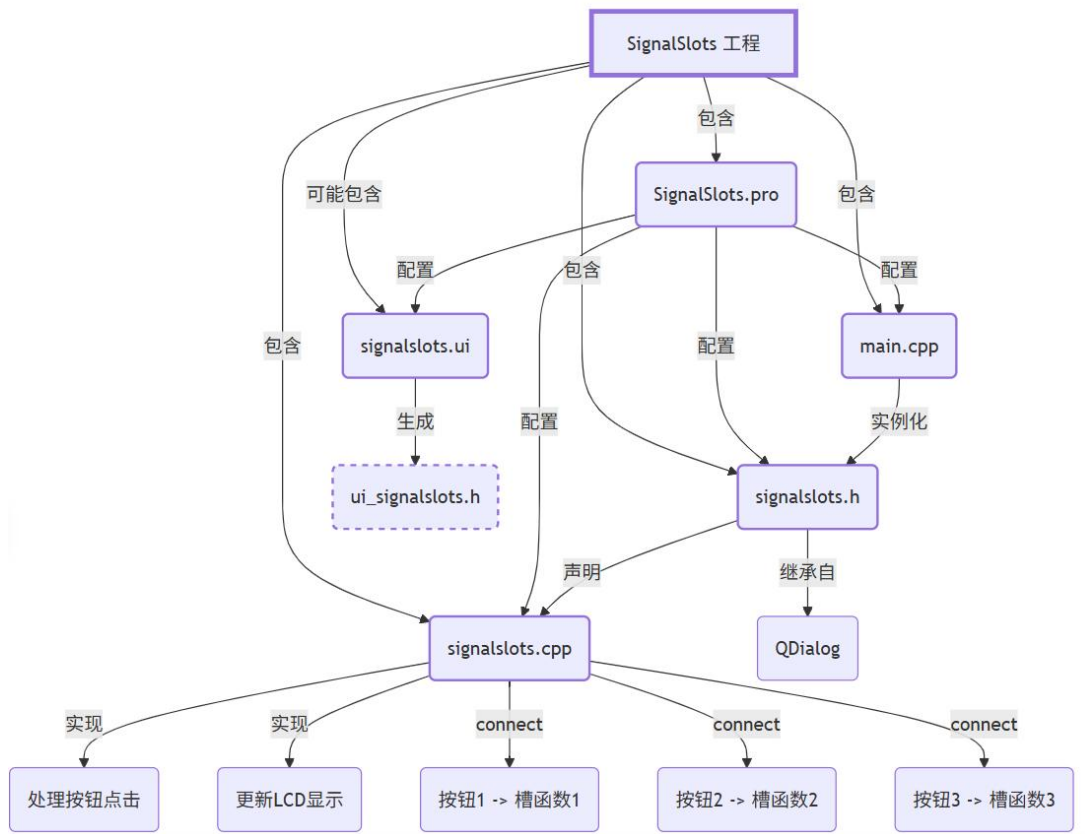
}
// 定时器槽：根据方向增减当前进度角度值并触发刷新
void Circularprogressbar::decreaseColorProgress()
{
    if(direction){
        currentColorProgress += 3.6; // 增长步长：一圈 360 / 3.6 = 100 次
        if(currentColorProgress > 360){
            currentColorProgress = 360; // 上限钳制
        }
    }else{
        currentColorProgress -= 1; // 回退更慢：需 360 次清空
        if(currentColorProgress < 0){
            currentColorProgress = 0;
            myTimer->stop(); // 清空后停止计时器节省资源
        }
    }
    update(); // 触发 repaint -> paintEvent
}
// 析构函数：目前无需额外处理；QTimer 由父对象自动回收
Circularprogressbar::~Circularprogressbar()
{
}

```

b) 信号与槽的应用（秒表）

i. 各文件关系

如图所示：



ii. 实验现象和原理

```

return
:SignalSlots)

this);
umber1建立信号槽连接
pushButton_1,
umber2建立信号槽连接
pushButton_2,
umber1.2.3建立信号槽连接
pushButton_3,
pushButton_3,
pushButton_3,

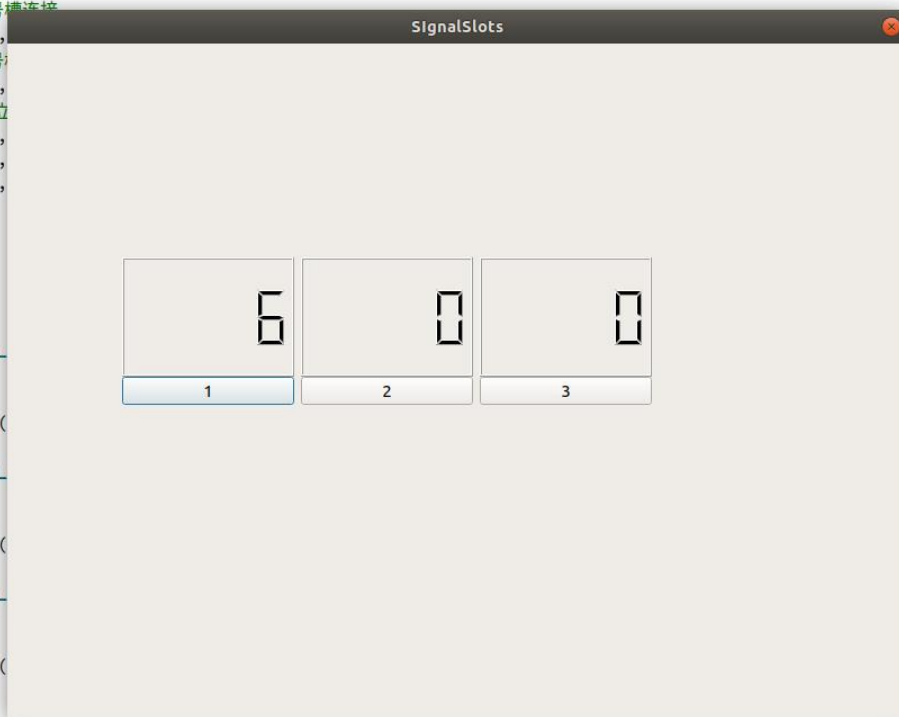
SignalSlots()

::pushbutton_1
r_1中数字加1
r_1->display()

::pushbutton_2
r_2中数字加1
r_2->display()

::pushbutton_3
r_3中数字加1
r_3->display()

```



用户界面含 3 个按钮与 3 个 QLCDNumber。

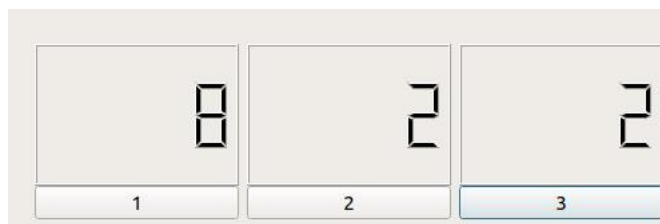
点击按钮 1: lcdNumber_1 数值 +1。

点击按钮 2: lcdNumber_2 数值 +1。

点击按钮 3: 由于对 pushButton_3 进行了三次 connect, 同一 clicked() 信号连到三个槽, 因此三个 LCD 各自递增一次。

```
20      // 按钮3 同一个 clicked() 信号连接到三个不同槽
21      // 点击一次会触发顺序调用三个槽函数, 各自递增对应 LCD
22      connect(ui->pushButton_3, SIGNAL(clicked()),
23              this, SLOT(pushbutton_1_Slot())); // lcd 1 +1
24      connect(ui->pushButton_3, SIGNAL(clicked()),
25              this, SLOT(pushbutton_2_Slot())); // lcd 2 +1
26      connect(ui->pushButton_3, SIGNAL(clicked()),
27              this, SLOT(pushbutton_3_Slot())); // lcd 3 +1
28
```

下图为继续点击 3 两次结果:



获取当前数字: QLCDNumber::value(); 更新显示: QLCDNumber::display(int)。

没有使用定时器, 更像按钮计数器而非真正秒表; 要实现秒表应该要加 QTimer 周期自增并开始/暂停。

iii. 核心代码文件注解

```
#include "signalslots.h"
#include "ui_signalslots.h"

// 构造函数: 初始化界面并建立信号槽连接
SignalSlots::SignalSlots(QWidget *parent)
    : QDialog(parent)
    , ui(new Ui::SignalSlots)
{
    ui->setupUi(this); // 由 Designer 生成的界面布置所有控件并赋给 ui->成员

    // SIGNAL/SLOT: 运行期字符串解析
    // 按钮 1 -> 槽 1: 单独递增 lcdNumber_1
    connect(ui->pushButton_1, SIGNAL(clicked()),
            this, SLOT(pushbutton_1_Slot()));

    // 按钮 2 -> 槽 2: 单独递增 lcdNumber_2
    connect(ui->pushButton_2, SIGNAL(clicked()),
```

```

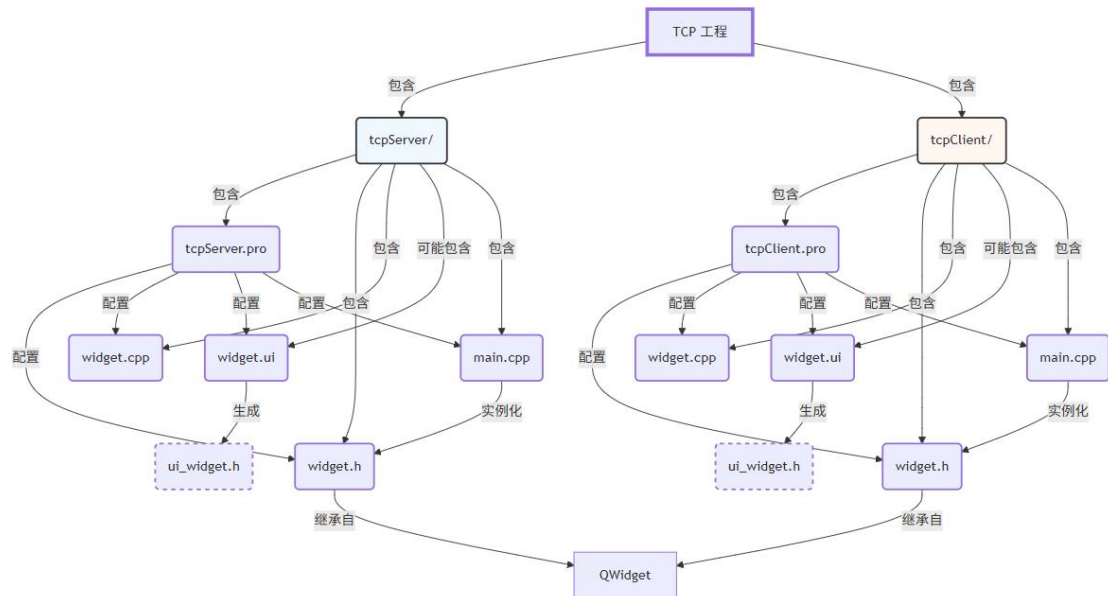
        this, SLOT(pushbutton_2_Slot()));
// 按钮 3 同一个 clicked() 信号连接到三个不同槽
// 点击一次会触发顺序调用三个槽函数，各自递增对应 LCD
connect(ui->pushButton_3, SIGNAL(clicked()),
        this, SLOT(pushbutton_1_Slot())); // lcd 1 +1
connect(ui->pushButton_3, SIGNAL(clicked()),
        this, SLOT(pushbutton_2_Slot())); // lcd 2 +1
connect(ui->pushButton_3, SIGNAL(clicked()),
        this, SLOT(pushbutton_3_Slot())); // lcd 3 +1
}
SignalSlots::~SignalSlots()
{
    delete ui; // 释放界面对象：其子控件由父子层级自动销毁
}
// 槽 1: lcdNumber_1 当前值 +1 后显示
void SignalSlots::pushbutton_1_Slot()
{
    ui->lcdNumber_1->display(ui->lcdNumber_1->value() + 1);
}
// 槽 2: lcdNumber_2 当前值 +1 后显示
void SignalSlots::pushbutton_2_Slot()
{
    ui->lcdNumber_2->display(ui->lcdNumber_2->value() + 1);
}
// 槽 3: lcdNumber_3 当前值 +1 后显示
void SignalSlots::pushbutton_3_Slot()
{
    ui->lcdNumber_3->display(ui->lcdNumber_3->value() + 1);
}

```

c) 网络编程 (TCP)

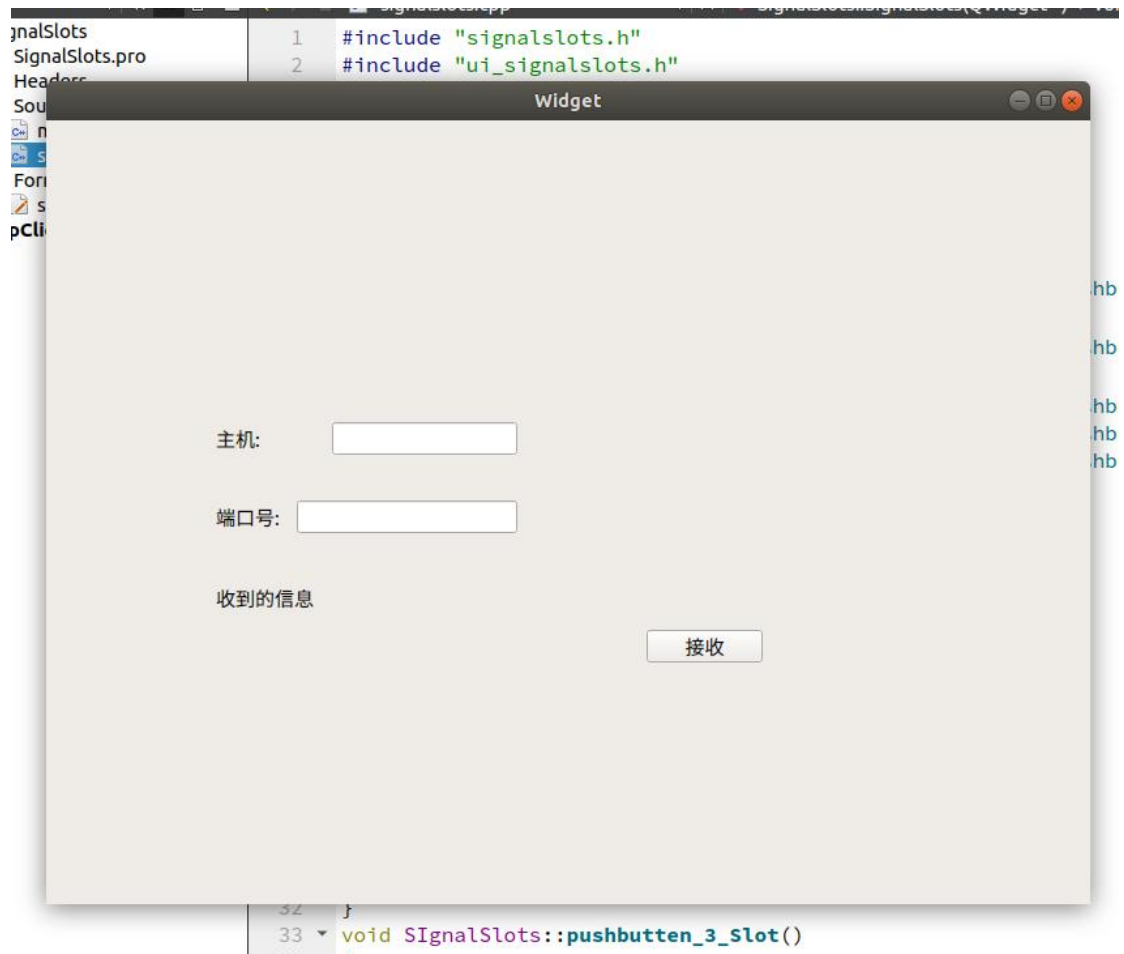
i. 各文件关系

如图所示



ii. 实验现象和原理

先启动客户端



再启动服务端，Server 界面会先显示等待连接。在 Client 输入对应的主机端口号，并点击接收，会触发下方这个函数的调用：

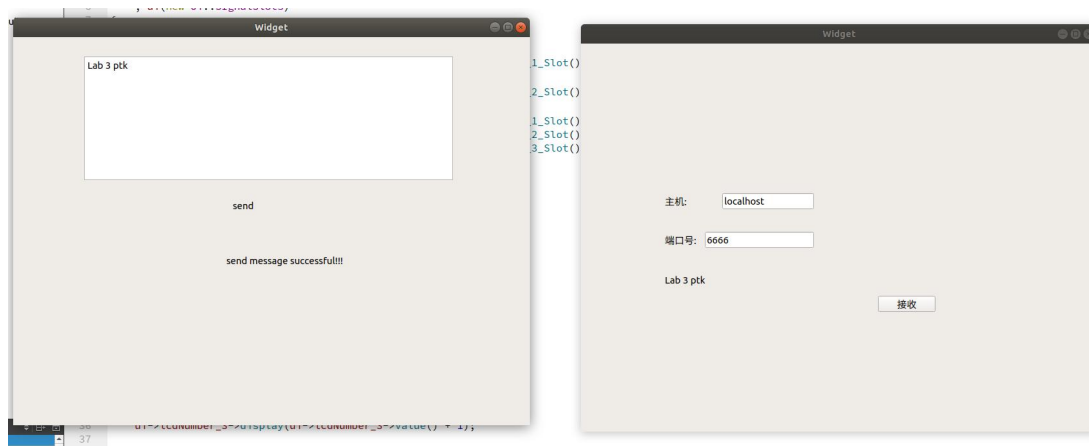
然后服务端收到连接请求，发射 newConnection()信号，触发 sendMessage()函数，发送字符。服务端从 textEdit_send 获取文本，通过 QDataStream 打包（消息头 + 消息体），发送给客户端套接字。

```

32 void Widget::sendMessage()
33 {
34     QByteArray block; // 暂存要发送的字节数组
35     QDataStream out(&block, QIODevice::WriteOnly); // 基于 block 创建数据流
36     out.setVersion(QDataStream::Qt_4_6); // 设置数据流版本（与客户端一致）
37
38     // 第一步：预留消息头空间（先写入 0，稍后回填实际长度）
39     out << (quint16) 0;
40
41     // 第二步：写入消息体（从文本框获取用户输入的字符串）
42     qDebug() << ui->textEdit_send->toPlainText(); // 调试输出
43     out << ui->textEdit_send->toPlainText(); // 序列化QString到流
44
45     // 第三步：回到流起始位置，写入消息体实际大小
46     out.device()->seek(0); // 移动到字节数组开头
47     out << (quint16)(block.size() - sizeof(quint16)); // 消息体长度=总长度-消息头 2 字节
48
49     // 获取已经建立的连接的客户端套接字
50     // nextPendingConnection() 返回队列中第一个等待处理的连接
51     QTcpSocket *clientConnection = tcpServer->nextPendingConnection();
52
53     // 连接断开时自动删除套接字对象（防止内存泄漏）
54     connect(clientConnection, SIGNAL(disconnected()),
55            clientConnection, SLOT(deleteLater()));
56
57     clientConnection->write(block); // 发送数据块到客户端
58     clientConnection->disconnectFromHost(); // 主动断开连接（短连接模式）
59
60     // 发送成功后更新界面状态提示
61     ui->statusLabel->setText("send message successful!!!");

```

接着 Client 套接字接收到数据，触发 readyRead()信号，调用 readMessage() 读取信息。就能看到下方显示从 Server 发来的字符串



结束阶段，服务端发送完成后调用 disconnectFromHost()，客户端触发 disconnected() 信号，套接字自动销毁 (deleteLater())。连接关闭。

iii. 核心代码文件注解

Server

```
#include "widget.h"
#include "ui_widget.h"

Widget::Widget(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::Widget)
{
    ui->setupUi(this); // 加载 Designer 界面布局

    tcpServer = new QTcpServer(this); // 创建 TCP 服务器对象，父对象管理内存

    // 监听本地主机的 6666 端口，等待客户端连接
    if(!tcpServer->listen(QHostAddress::LocalHost, 6666))
    {
        // 监听失败：可能端口被占用或权限不足
        qDebug() << "error" << tcpServer->errorString();
        close(); // 关闭窗口退出程序
    }

    // 当有新客户端连接时，QTcpServer 发射 newConnection() 信号
    // 连接到 sendMessage() 槽，自动发送数据
    connect(tcpServer, SIGNAL(newConnection()),
            this, SLOT(sendMessage()));
}

Widget::~Widget()
{
    delete ui; // 释放界面对象；tcpServer 由父对象自动销毁
}

// 槽函数：有新连接时立即发送数据
void Widget::sendMessage()
{
    QByteArray block; // 暂存要发送的字节数组
    QDataStream out(&block, QIODevice::WriteOnly); // 基于 block 创建数据流
    out.setVersion(QDataStream::Qt_4_6); // 设置数据流版本（与客户端一致）

    // 第一步：预留消息头空间（先写入 0，稍后回填实际长度）
    out << (quint16) 0;

    // 第二步：写入消息体（从文本框获取用户输入的字符串）
    qDebug() << ui->textEdit_send->toPlainText(); // 调试输出
```

```

out << ui->textEdit_send->toPlainText();    // 序列化 QString 到流

// 第三步：回到流起始位置，写入消息体实际大小
out.device()->seek(0);                      // 移动到字节数组开头
out << (quint16)(block.size() - sizeof(quint16)); // 消息体长度=总长度-消息头 2 字节

// 获取已经建立的连接的客户端套接字
// nextPendingConnection() 返回队列中第一个等待处理的连接
QTcpSocket *clientConnection = tcpServer->nextPendingConnection();

// 连接断开时自动删除套接字对象（防止内存泄漏）
connect(clientConnection, SIGNAL(disconnected()),
        clientConnection, SLOT(deleteLater()));

clientConnection->write(block);              // 发送数据块到客户端
clientConnection->disconnectFromHost();     // 主动断开连接（短连接模式）

// 发送成功后更新界面状态提示
ui->statusLabel->setText("send message successful!!!");
QDebug() << "send message successful!!!";
}

```

Client

```

#include "widget.h"
#include "ui_widget.h"

Widget::Widget(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::Widget)
{
    ui->setupUi(this); // 加载 Designer 界面布局

    tcpSocket = new QTcpSocket(this); // 创建客户端套接字，父对象管理内存

    // 数据到达时触发 readMessage() 槽解析消息
    connect(tcpSocket, SIGNAL(readyRead()),
            this, SLOT(readMessage()));

    // 连接失败/断开等错误触发 displayError() 槽输出诊断信息
    connect(tcpSocket, SIGNAL(error(QAbstractSocket::SocketError)),
            this, SLOT(displayError(QAbstractSocket::SocketError)));

    // 连接按钮点击触发连接逻辑
    connect(ui->pushButton, SIGNAL(clicked()),
            this, SLOT(pushButton_clicked()));
}

```

```

}

Widget::~Widget()
{
    delete ui; // 释放界面对象, tcpSocket 由父对象自动销毁
}

// 发起到服务器的连接
void Widget::newConnect()
{
    blockSize = 0; // 重置消息块大小, 准备接收新数据
    tcpSocket->abort(); // 强制关闭现有连接并清空缓冲区

    // 从界面获取主机地址 (IP 或域名) 和端口号
    // 异步连接: 成功后触发 connected() 信号 (未连接), 失败触发 error()
    tcpSocket->connectToHost(ui->hostLineEdit->text(),
                            ui->portLineEdit->text().toInt());
}

// 接收并解析服务器发送的消息
void Widget::readMessage()
{
    QDataStream in(tcpSocket); // 基于套接字创建数据流
    in.setVersion(QDataStream::Qt_4_6); // 必须与服务端版本一致

    // 第一阶段: 读取消息体大小 (quint16, 2 字节)
    if(blockSize == 0) // 如果是首次接收或新消息开始
    {
        // 检查是否至少收到 2 字节 (消息头)
        if(tcpSocket->bytesAvailable() < (int)sizeof(quint16))
            return; // 数据不足, 等待下次 readyRead() 触发

        in >> blockSize; // 读取后续消息体的大小 (不包含这 2 字节本身)
    }

    // 第二阶段: 等待接收完整消息体
    if(tcpSocket->bytesAvailable() < blockSize)
        return; // 数据未到齐, 继续等待

    // 完整接收后解析消息内容
    in >> message; // 从流中读取 QString

    ui->messageLabel->setText(message); // 在界面标签上显示接收到的消息

    // 注意: 此处未重置 blockSize, 若需接收下一条消息应在合适位置重置
}

// 错误处理槽: 输出套接字错误信息

```



```

void Widget::displayError(QAbstractSocket::SocketError)
{
    // 典型错误: ConnectionRefusedError (服务器未运行)、

    qDebug() << "error" << tcpSocket->errorString();
}

// 连接按钮点击槽: 触发连接请求
void Widget::pushButton_clicked()
{
    newConnect(); // 调用连接函数
}

clientConnection->disconnectFromHost();
// 发送数据成功后, 显示提示
ui->statusLabel->setText("send message successful!!!");
qDebug() <<"send message successful!!!";
}

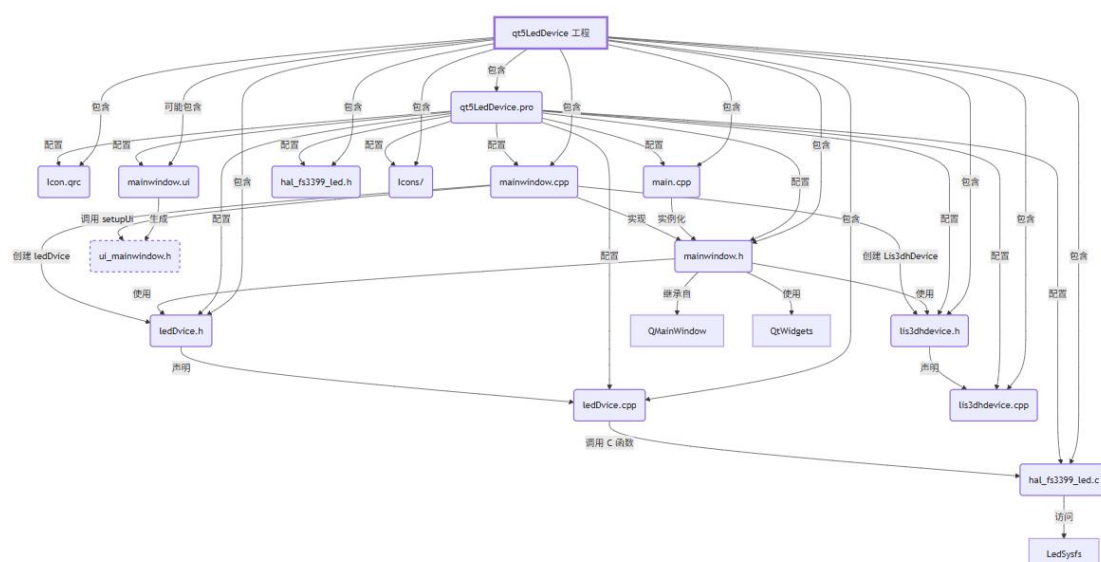
```

2. 针对第二次 Qt 实验的 8 个实验样例:

a) LED 灯

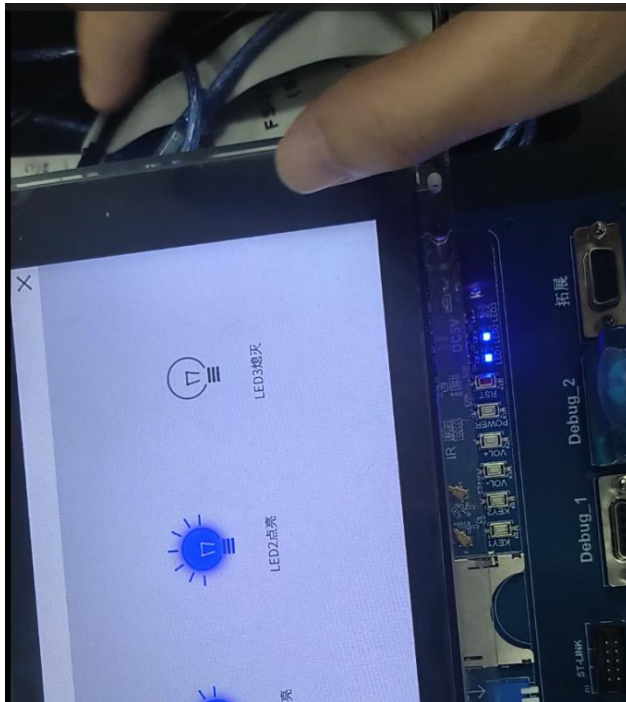
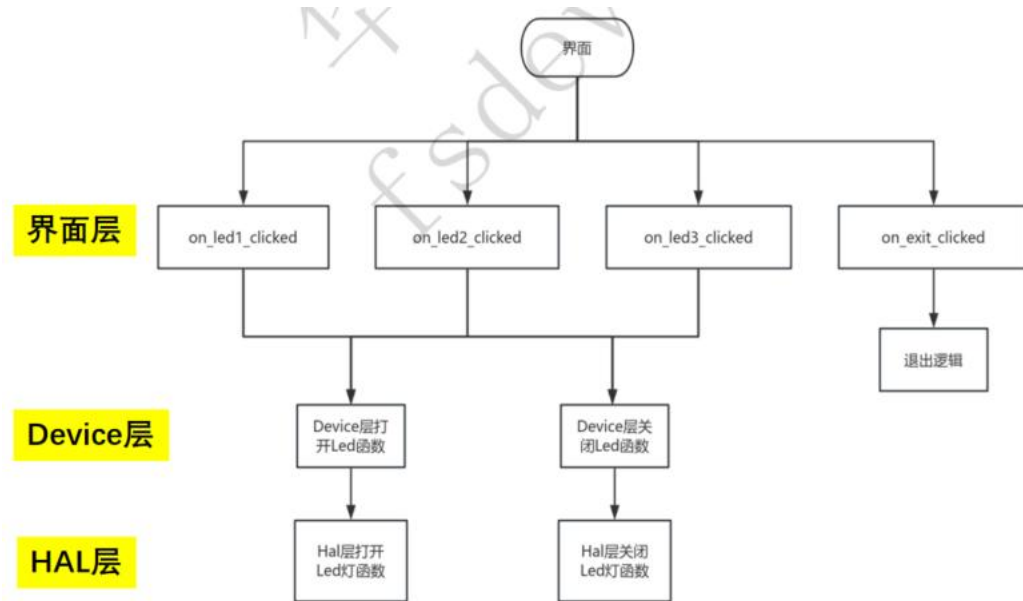
i. 各文件关系

如图所示:



ii. 实验现象和原理(结合注解后的核心代码文件)

这里按照指导书的逻辑分层图分析原理



初始界面，在界面层使用 QPushButton、QHBoxLayout 搭建界面，初始化 UI 组件，连接信号槽，创建线程。

MainWindow.cpp:

整体样式的绘制。

```

7     QGraphicsScene *scene = new QGraphicsScene(this);
8     view = new QGraphicsView(scene, this);
9     customWidget = new CustomWidget(); // 创建自定义Widget
10
11     QScreen *screen = QApplication::primaryScreen(); // 获取当前屏幕
12     QRect screenGeometry = screen->geometry(); // 获取屏幕尺寸
13     int screenWidth = screenGeometry.width(); // 屏幕宽度
14     int screenHeight = screenGeometry.height(); // 屏幕高度
15
16     customWidget->resize(screenWidth, screenHeight); // 设置自定义Widget的大小为全屏
17     customWidget->showFullScreen(); // 显示为全屏
18     customWidget->topLabel->raise(); // 将顶部标签置于顶层
19
20     view->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff); // 水平滚动条禁用
21     view->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOff); // 垂直滚动条禁用
22     proxyWidget = scene->addWidget(customWidget); // 将自定义Widget添加到场景中
23     proxyWidget->setTransformOriginPoint(proxyWidget->boundingRect().center()); // 设置变换原点
24     proxyWidget->setZValue(100); // 设置Z值为100

```

布局：

```

// 设置 QGraphicsView 为中心控件或根据需要进行布局
setCentralWidget(view);

```

调用封装的信号与槽：

```

67     connect(customWidget->m_ledOnButton7, SIGNAL(clicked()), this, SLOT(on_exit_clicked()));
68     connect(customWidget->m_ledOnButton1, SIGNAL(clicked()), this, SLOT(on_led1_clicked()));
69     connect(customWidget->m_ledOnButton2, SIGNAL(clicked()), this, SLOT(on_led2_clicked()));
70     connect(customWidget->m_ledOnButton3, SIGNAL(clicked()), this, SLOT(on_led3_clicked()));

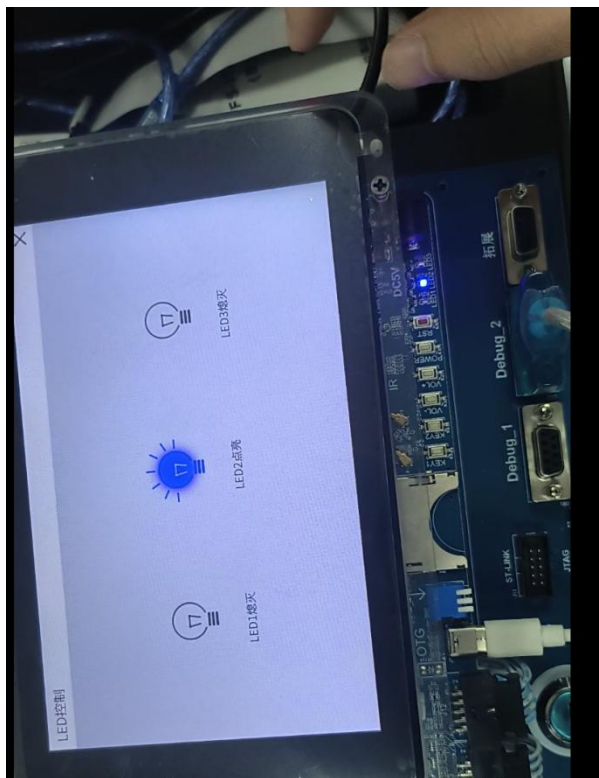
```

线程、信号与槽的控制：

```

26     lis3dh = new Lis3dhDevice(); // 创建Lis3dhDevice对象
27     thread3dh = new QThread(); // 创建新线程
28     lis3dh->moveToThread(thread3dh); // 将lis3dh对象移至新线程
29     thread3dh->start(); // 启动线程
30
31     // 连接信号与槽，当线程开始时，运行lis3dh的run函数
32     connect(thread3dh, &QThread::started, lis3dh, &Lis3dhDevice::run);
33
34     // 连接停止线程信号，确保线程安全退出
35     connect(lis3dh, &Lis3dhDevice::stopthread, [&]() {
36         if(thread3dh->isRunning()){
37             lis3dh->changeRunningState(false); // 改变运行状态为false
38             thread3dh->quit(); // 退出线程
39         }

```



当点击 led 灯图标，信号槽被触发，在 MainWindow 的 on_led1/2/3_clicked () 中调用调用 ledDvice->setLedState(true)。

MainWindow.cpp:

```
76     LED按钮点击槽函数
77     id MainWindow::on_led1_clicked(){
78         if(!customWidget->Led1State){
79             customWidget->m_hqled->ledOn(LED1); // 打开LED1点亮
80             customWidget->Led1->setText("LED1点亮"); // 设置标签文本为LED1点亮
81             customWidget->m_ledOnButton1->setIcon(QIcon(":/icon/Icons/ledblue.png")); // 设置按钮图标为红灯
82             customWidget->Led1State = true; // 更新Led1状态为打开
83             return;
84         } else {
85             customWidget->m_hqled->ledOff(LED1); // 关闭LED1点亮
86             customWidget->Led1->setText("LED1熄灭"); // 设置标签文本为LED1熄灭
87             customWidget->m_ledOnButton1->setIcon(QIcon(":/icon/Icons/ledoff.png")); // 设置按钮图标为关闭状态
88             customWidget->Led1State = false; // 更新LED1状态为关闭
89         }
89     }
```

接着 Device 层执行， ledDvice::setLedState() 调用 hal 层的 ledOn()或 ledOff。

```

17 int ledDvice::ledOn(int nr)
18 {
19     int ret = 0;
20     ret = led_on(nr);
21     if(ret<0)
22     {
23         perror("open led failed\n");
24     }
25     return ret;
26 }
27
28
29 // 关闭指定编号的LED灯
30 int ledDvice::ledOff(int nr)
31 {
32     int ret = 0;
33     ret = led_off(nr);
34     if(ret<0)
35     {
36         perror("close led failed\n");
37     }
38     return ret;
39 }
40

```

在 HAL 层，通过 ioctl 接口向 led 物理路径写入状态（0/1），从物理层面控制亮灭。
Hal_fs3399_led.c

```

3  int led_init()
4  {
5      fd = 0;
6      fd = open(LED_PATH, O_RDWR);
7      if(fd < 0){
8          perror("open led failed");
9          return -1;
10     }
11     ioctl(fd, LED1_OFF);
12     ioctl(fd, LED2_OFF);
13     ioctl(fd, LED3_OFF);
14     return 0;
15 }

```

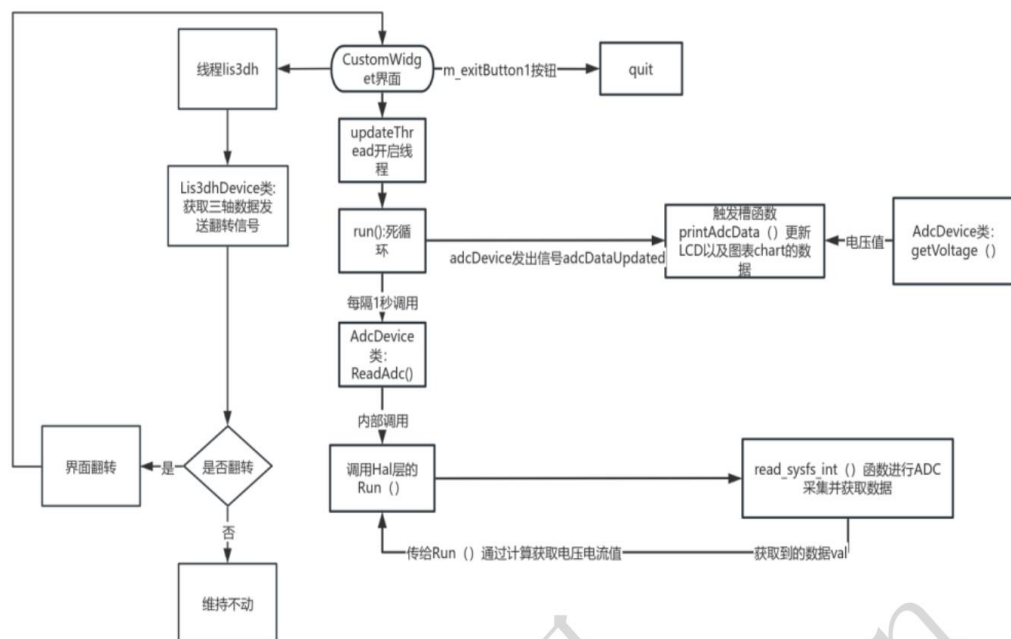
```

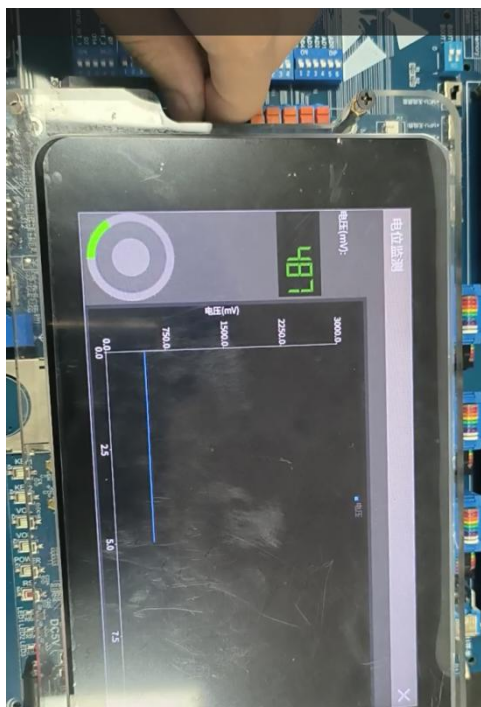
17 // 打开指定编号的LED灯
18 int led_on(int nr)
19 {
20
21     // 控制LED
22     switch (nr) {
23     case LED1:
24         ioctl(fd, LED1_ON);
25         break;
26     case LED2:
27         ioctl(fd, LED2_ON);
28         break;
29     case LED3:
30         ioctl(fd, LED3_ON);
31         break;
32     default:
33         break;
34     }
35     return 0;
36 }

```

b) 电位检测 (ADC)

这里按照指导书的流程图，结合代码和现象说明原理





启动 app 后，MainWindow 先进行了初始化，创建 AdcDevice 对象和 UpdateThread 进程，采用 QChart 类以折线图的形式和 QLCD 的形式以及自定义圆环的形式呈现当前开发板的电压值，然后采用 QGridLayout 网格布局对界面进行布局（界面层），同时也会使用到 QGraphicsScene 以及 QGraphicsView 类来实现界面的旋转。通过调用 Device 层以及 Hal 层的接口达到获取 adc 状态的目的。



上方我通过旋转电压旋钮，控制电压大小变化，可以从折线图看到变化轨迹。
具体的电压变化接收原理，是通过 UpdateThread::run() 循环执行：调用 AdcDevice::ReadAdc()。ReadAdc () 再调用 hal 层的 Run ()。
adcdevice.cpp:

```
7
8 // 基类的ReadAdc函数
9 void AdcDevice::ReadAdc()
10 {
11     int ret = Run(); // 执行ADC读取操作
12
13     if(ret < 0) // 判断读取是否成功
14     {
15         qDebug()<<"read failed"; // 如果读取失败，输出错误信息
16     }
17     else
18     {
19         Vol = ret;
20     }
21 }
```

Run () 的代码如下。

Hal_fs3399_adc.c

```
63 //在Run函数中读取数据
64 int Run()
65 {
66     int data = 0;
67     voltage = read_sysfs_int(data);
68     //printf("vol:%d\n",voltage);
69     return voltage;
70 }
```

Run()中调用了 read_sysfs_int，它的实现又如下。

hal_fs3399_adc.c


```

44 //选择通道读取数据
45 int read_sysfs_int(int val)
46 {
47     adc_init(); // 每次读取前重新初始化
48     char data[20] = {0}; // 缓冲区：存储读取的字符串
49
50     // 从文件描述符读取数据
51     int err = read(fd, data, sizeof(data));
52     if (err > 0) {
53         // 读取成功：将字符串转换为整数
54         int num_data;
55         int sscanf_result = atoi(data);
56         // val = sscanf_result * 1.8;
57         val = sscanf_result;
58         usleep(1000000); // 延时1秒
59         return val;
60     } else {
61         return -1;
62     }
63     adc_close();
64 }
65

```

读取完之后再发射 adcDataUpdated() 信号，休眠一秒。

Adcdevice.h:

```

41 protected:
42     void run() {
43         qDebug() << "run";
44         while (true) {
45             adcDevice->ReadAdc(); // 读取ADC数据
46             emit adcDevice->adcDataUpdated(); // 发射数据更新信号
47             msleep(1000); // 每秒更新一次
48         }
49     }
50 };
51
52 #endif // ADCDEVICE_H
53

```

最后 MainWindow::printAdcData() 槽函数响应：获取电压值 getVoltage(); 更新图表显示 (QLineSeries); 更新 CircularProgressBar (这里就是跟实验一用到的自定义进度条组件差不多) 显示。

mainwindow.cpp:

```

80
81 //获取到数据之后更新图表
82 void MainWindow::printAdcData()
83 {
84     // 从 ADC 对象读取当前电压值
85     const float voltageValues = customWidget->myadc->getVoltage();
86     // 在 LCD 显示器上显示电压值
87     customWidget->voltageLCD->display(voltageValues);
88     // 更新自定义进度条的电压显示
89     customWidget->voltageProgressBar->setVoltage(voltageValues);
90     // 将 (x, 电压) 数据点追加到图表序列
91     customWidget->voltageSeries->append(customWidget->x, voltageValues);
92     customWidget->x++;
93
94     if (customWidget->x >= 10) {
95         customWidget->chart->scroll(1, 0);
96         // 设置 X 轴范围为最新的 10 个点
97         customWidget->axisX->setRange(customWidget->x - 9, customWidget->x);
98     }
99 }

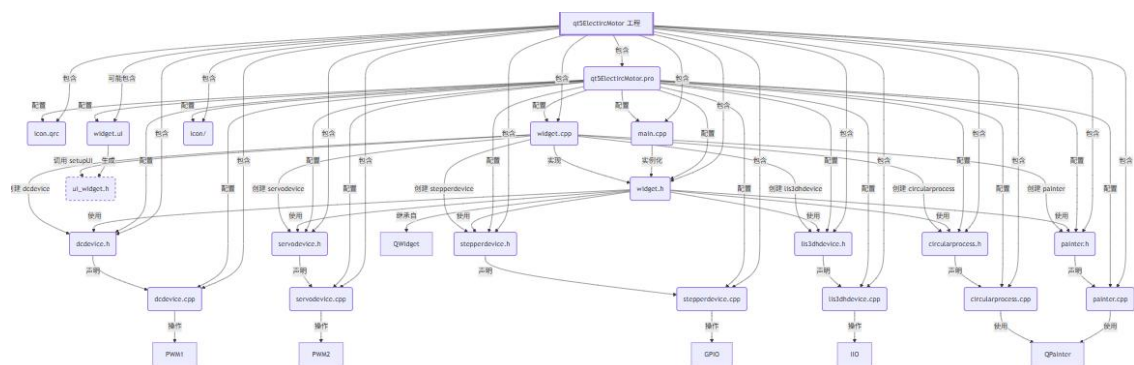
```

c) 电机综合 (3 个设备)

i. 各文件关系

文件树:

各文件实现和依赖关系图:



ii. 实验现象和原理(结合注解后的核心代码文件)

实验现象:

界面的绘制在 widget 中, 与 LED 基本一致。核心区别就在连接信号、槽的部分, 以及线程的设置。几个电机的初始化、关闭也都在这里定义。并且当界面切换时, 也会触发当前电

机线程的关闭和开启新电机的线程。

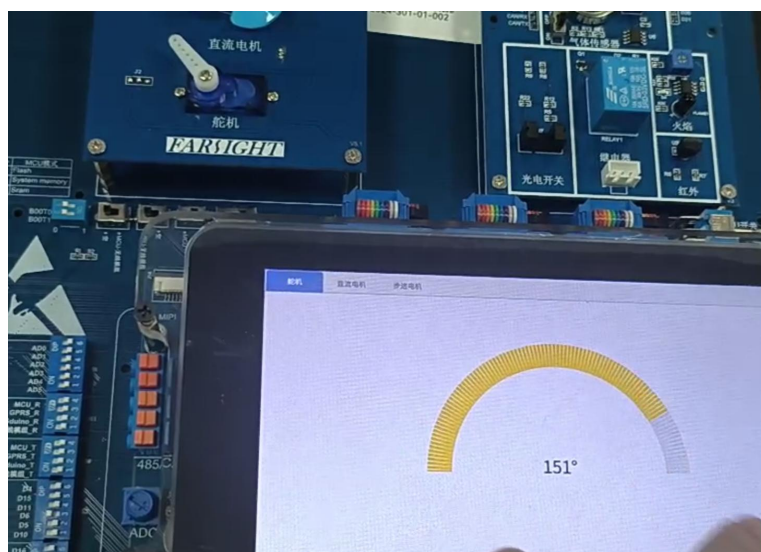
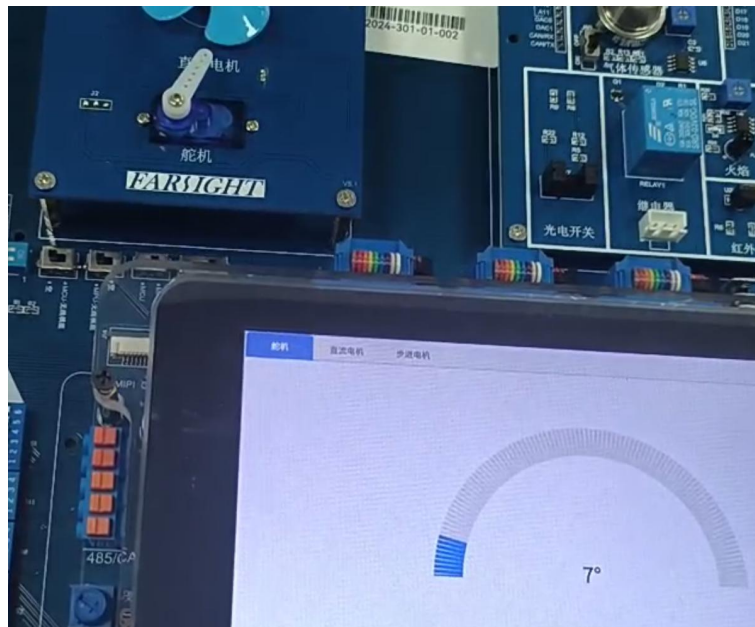
```
77 // 连接顶部退出按钮的点击信号到退出槽函数：退出程序与设备清理
78 connect(customWidget->topexit,&QPushButton::clicked,this,&Widget::exitSlot);
79
80 // 圆形进度/角度控件 angleChanged 信号触发舵机转动（百分比→角度）
81 connect(customWidget->round1,&CircularProcess::angleChanged,this,&Widget::trunServo);
82
83 // 直流电机三个功能按钮：返回(低速/后退?)、停止、启动/前进
84 connect(customWidget->fanBackButton,&QPushButton::clicked,this,&Widget::backSlot);
85 connect(customWidget->fanStopButton,&QPushButton::clicked,this,&Widget::stopFanSlot);
86 connect(customWidget->fanButton,&QPushButton::clicked,this,&Widget::fanButtonSlot);
87
88 // 定时器驱动风扇动画控件刷新（旧式 SIGNAL/ SLOT 写法）
89 connect(customWidget->timer,SIGNAL(timeout()),customWidget->fan,SLOT(update()));
90
91 // 三个自定义信号 Set0/Set1/Set2 用于切换风扇显示的“分区”或状态
92 connect(this, SIGNAL(Set0()), customWidget->fan, SLOT(setPart0()));
93 connect(this, SIGNAL(Set1()), customWidget->fan, SLOT(setPart1()));
94 connect(this, SIGNAL(Set2()), customWidget->fan, SLOT(setPart2()));
95
96 // 步进电机专用线程启动时调用 stepperDevice::run 进入其循环逻辑
97 connect(customWidget->thread,&QThread::started,customWidget->stepper,&stepperDevice::run);
98
99 // 线程结束时复位运行标志 isrun=false (lambda 捕获 = 保持当前指针)
100 connect(customWidget->thread,&QThread::finished,this,[=]() {
101     | customWidget->stepper->isrun = false;
102 });
```

```
104 // 遍历 Page3 页面下所有子对象，找到 objectName 以 "part" 开头的按钮做统一绑定
105 foreach(QObject *obj, customWidget->Page3->children()) {
106     QPushButton *button = qobject_cast<QPushButton*>(obj);
107     if(button)
108         if (button && button->objectName().startsWith("part")) {
109             // 从名称提取索引：如 "part2" → 2
110             QString buttonName = button->objectName();
111             int partIndex = buttonName.mid(4).toInt();
112
113             // 点击按钮后根据 partIndex 控制步进电机线程与分区设置
114             connect(button, &QPushButton::clicked, this, [&, partIndex]() {
115                 // 非0分区且线程未运行 → 启动线程
116                 if (partIndex != 0 && !customWidget->thread->isRunning()) {
117                     customWidget->thread->start();
118                 }
119                 // 分区0且线程在运行 → 请求退出线程
120                 else if(partIndex == 0 && customWidget->thread->isRunning()){
121                     customWidget->thread->quit();
122                 }
123                 // 设置步进电机的分区/模式（内部根据索引切换不同逻辑）
124                 customWidget->stepper->setPart(partIndex);
125             });
126         }
127     }
```

对于三种不同电机的硬件控制原理解析：

Device 层直接与各个电机的设备文件进行交互，故该工程没有编写 HAL 层的代码

(1) 舵机



拖动圆形控件，触发角度百分比变化。

Widget.cpp:

```
80 // 圆形进度/角度控件 angleChanged 信号触发舵机转动（百分比→角度）
81 connect(customWidget->round1,&CircularProcess::angleChanged,this,&Widget::trunServo);
```

若角度变化，就触发设置舵机到指定角度。

具体实现在 device 层，通过 ioctl 调用直接控制舵机角度。

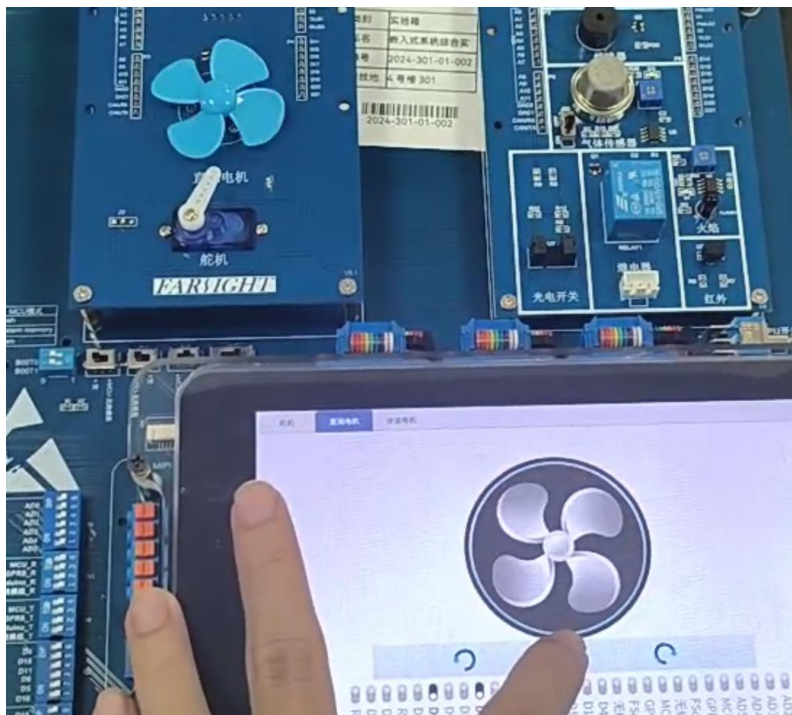
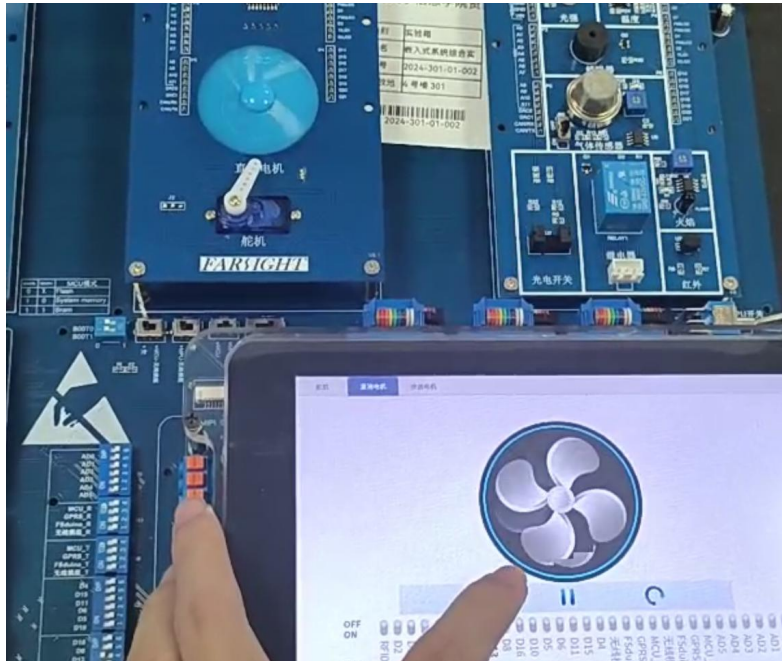
Servodevice.cpp:


```

19 // 控制舵机角度：通过 ioctl 调用
20 void servoDevice::steerServo(int percent)
21 {
22     if (percent >= 0 && percent <= 180) // 检查输入的角度是否在有效范围内
23     {
24         if (ioctl(fd, SET_ANGLE, percent) < 0) // 发送控制命令
25         {
26             qDebug() << "ioctl SET_ANGLE failed"; // 如果命令发送失败，输出调试信息
27         }
28     }
29 }

```

(2) 直流电机



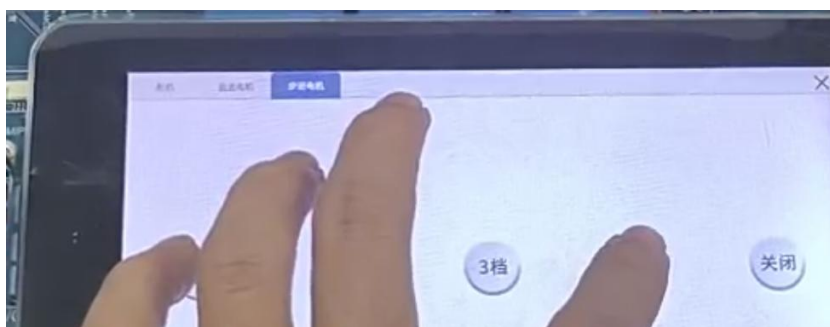
在 Widget 中调用封装的函数，传入点击的响应参数。左到右依次是逆时针转、停止、顺时针转。

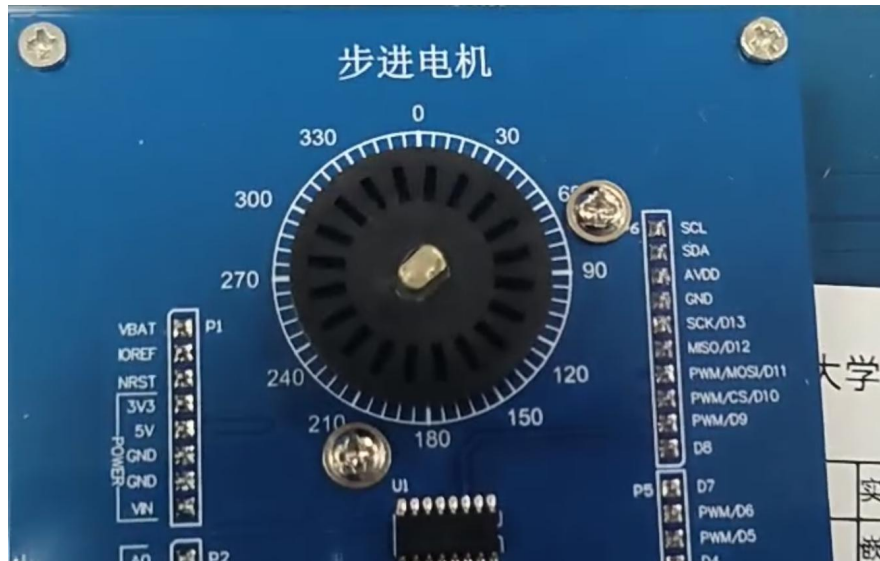
```
82
83 // 直流电机三个功能按钮：逆时针、停止、顺时针
84 connect(customWidget->fanBackButton,&QPushButton::clicked,this,&Widget::backSlot);
85 connect(customWidget->fanStopButton,&QPushButton::clicked,this,&Widget::stopFanSlot);
86 connect(customWidget->fanButton,&QPushButton::clicked,this,&Widget::fanButtonSlot);
87
88 // 定时器驱动风扇动画控件刷新
89 connect(customWidget->timer,SIGNAL(timeout()),customWidget->fan,SLOT(update()));
90
91 // 三个自定义信号 Set0/Set1/Set2 用于切换风扇显示的状态
92 connect(this, SIGNAL(Set0()), customWidget->fan, SLOT(setPart0()));
93 connect(this, SIGNAL(Set1()), customWidget->fan, SLOT(setPart1()));
94 connect(this, SIGNAL(Set2()), customWidget->fan, SLOT(setPart2()));
95
```

Device 层也是通过 ioctl 实现直接控制直流电机：

```
19 // 控制直流电机
20 void dcDevice::controlDc(int operation)
21 {
22     switch (operation) { // 根据传入的操作编号执行不同的控制命令
23     case 0:
24         ioctl(dcfid, DC_MOTOR_ON); // 打开电机
25         ioctl(dcfid, DC_MOTOR_DIR, &operation); // 设置电机方向
26         break;
27     case 1:
28         ioctl(dcfid, DC_MOTOR_ON); // 打开电机
29         ioctl(dcfid, DC_MOTOR_DIR, &operation); // 设置电机方向
30         break;
31     case 2:
32         ioctl(dcfid, DC_MOTOR_OFF); // 关闭电机
33         break;
34     }
35 }
```

(3) 步进电机





Widget 中相关部分，定义了相关按钮的逻辑：

```

96 // 步进电机专用线程启动时调用 stepperDevice::run 进入其循环逻辑
97 connect(customWidget->thread,&QThread::started,customWidget->stepper,&stepperDevice::run);
98
113 // 点击按钮后根据 partIndex 控制步进电机线程与分区设置
114 connect(button, &QPushButton::clicked, this, [&, partIndex]() {
115     // 非0分区且线程未运行 → 启动线程
116     if (partIndex != 0 && !customWidget->thread->isRunning()) {
117         customWidget->thread->start();
118     }
119     // 分区0且线程在运行 → 请求退出线程
120     else if(partIndex == 0 && customWidget->thread->isRunning()){
121         customWidget->thread->quit();
122     }
123     // 设置步进电机的模式
124     customWidget->stepper->setPart(partIndex);
125 }

```

通过 Device 层更改步进电机的状态：


```
115 // 更改步进电机的运行状态
116 void stepperDevice::changeRunningState(bool state)
117 {
118     isrun = state; // 根据传入的状态设置运行标志
119 }
120
121 // 关闭步进电机
122 void stepperDevice::closeStepper()
123 {
124     printf("stepperfd:%d\n", stepperfd); // 输出步进电机文件描述符
125     int ret = 0;
126     if (stepperfd > 0) {
127         ret = close(stepperfd); // 尝试关闭文件描述符
128         if (ret < 0) {
129             perror("closeStepper\n"); // 关闭失败时输出错误信息
130         }
131         stepperfd = 0; // 重置文件描述符
132     }
133 }
```